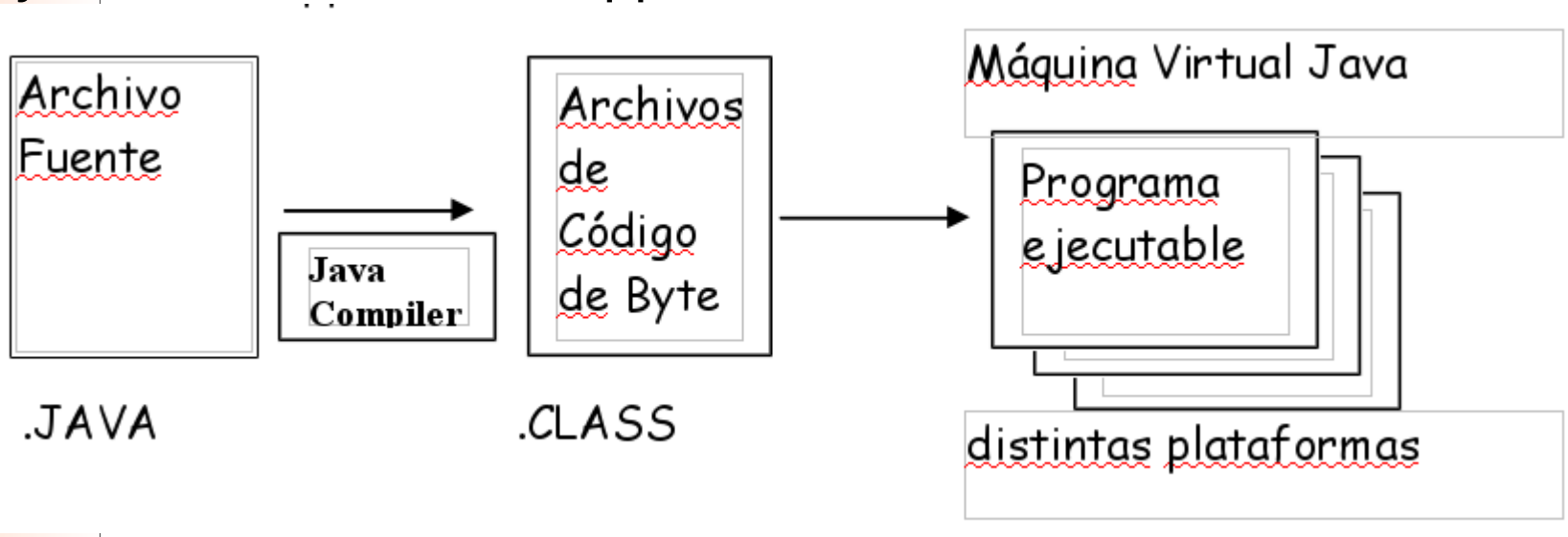
El Lenguaje de programación Java

Características

- Java es un lenguaje de programación Orientado a Objetos, concurrente con soporte para el modelo cliente/servidor.
- Las bases de programación estructurada son evidentes como en C++.
- Sus capacidades OO NO se añadieron después de creado (como en C++), por ello es más simple
- Java es un lenguaje seguro.
- Brindaba un mecanismo de programación en la WWW.
- Posee un mecanismo de manejo de excepciones fácil de utilizar, que garantiza consistencia en los programas.
- Provee capacidad para programación concurrente mediante el concepto de “Multithreading”.

El Lenguaje de programación Java

- Java simula una máquina virtual:
- Cuando Java compila un programa, no se genera un ejecutable en código máquina, sino “códigos de byte de Java” (CBJ).
- Los códigos de byte son instrucciones escritas para una máquina virtual (JVM).
- Para la ejecución del programa se debe tener un intérprete de CBJ.
- Se simula una JVM en cualquier computadora y SO que se use.
- El explorador es capaz de interpretar los CBJ.
- Con la JVM se consigue: independencia de la máquina, seguridad y reducido tamaño de applets.



El Lenguaje de programación Java

- Java contiene sus propias librerías llamadas “packages” que son independientes de la plataforma donde se ejecuten.
- De la misma forma, por medio de la JVM se consigue independizarse de si el producto final correrá en un Pentium Intel, un Sun SPARC, etc., ya que el compilador no genera instrucciones nativas del procesador, sino códigos de bytes.
- Si la JVM no existe en el sentido real: Cómo se ejecuta el código JAVA?
- Sun Microsystems implementó una versión de la JVM para cada plataforma.
- Al cargar el archivo de bytecodes (*class file*) éste se ejecuta en la JVM de esa máquina.
- La JVM lee la class file y hace el trabajo especificado.
- El Software Development Kit (SDK) incluye un intérprete JAVA que implementa la JVM para una plataforma.

El Lenguaje de programación Java

JAVA ES ORIENTADO A OBJETOS
Java fue desarrollado por programadores C++ de Sun Microsystems.
Copiaron la sintaxis de C++ y incluyendo su capacidad OO. Se identifican y describen clases y se escribe el código para los métodos.

JAVA SOPORTA EL MODELO CLIENTE/SERVIDOR
Java expandió la librería estándar de C++ para incluir comunicación en red.
Esto permite escribir aplicaciones para Internet.

El Lenguaje de programación Java

OO Java

El lenguaje Java asume que se está utilizando Programación OO; por lo tanto en Java “todo es un objeto” (Eckel). Al tratar todo como objetos, se utiliza siempre una sintaxis consistente.

En realidad, el identificador que se manipula es una “referencia” a un objeto. Java ofrece referencias en lugar de punteros (como C o C++), porque son más seguros y fáciles de manejar.

Si queremos almacenar una frase, creamos una referencia a un String, de la forma:

```
String s;   // se declara una referencia a un objeto String
```

Se ha creado una referencia, no un objeto. Por lo tanto, no se le podría enviar ningún mensaje.

El Lenguaje de programación Java

Cuando se crea una referencia, se la conecta con un objeto, por medio de la palabra clave “new”. En el ej. anterior, sería:

```
String s = new String("programación");
```

Si bien Java provee varios tipos además de String, lo importante es que de la misma forma se pueden crear tipos propios del usuario.

El Lenguaje de programación Java

Almacenamiento de Datos en Java

Hay seis diferentes lugares donde pueden almacenarse los datos:

1. Registers:

- El medio más rápido de almacenamiento, registros dentro del procesador
- Su número es limitado.
- No se tiene control directo de los mismos.

2. La pila (stack):

- Está dentro de la memoria RAM, el procesador la controla vía el Stack Pointer. El SP se mueve hacia abajo para asignar nueva memoria y hacia arriba para liberarla. Es un medio rápido y eficiente de alocación de memoria.
- El compilador debe conocer el tamaño exacto y tiempo de vida de todos los datos almacenados en la pila, mientras esté creando el programa.
- Esta restricción limita la flexibilidad: se almacenan las referencias a los objetos en la pila, no los objetos mismos.

El Lenguaje de programación Java

3. El montículo (heap):

- Es una memoria de propósito general dentro de la RAM, donde viven todos los objetos.
- La ventaja respecto a la pila es que el compilador no necesita conocer cuánta memoria asignar desde el heap o cuánto tiempo estará en el heap.
- Cuando se necesita un objeto, se lo crea usando new y el almacenamiento se asigna en el montículo.
- Precio para la flexibilidad: toma más tiempo asignar almacenamiento en el heap que en la pila.

4. Almacenamiento estático.

- Estático se usa en el sentido de “en una locación fija” (también en la RAM)
- Contiene datos que están disponibles durante todo el tiempo que el programa corre.
- Los objetos de Java nunca se alojan en el almacenamiento estático.

El Lenguaje de programación Java

5. Almacenamiento de constantes.

- Se almacenan directamente en el código del programa, el cual es seguro porque nunca cambia.

6. Almacenamiento No-RAM

- Los datos pueden existir mientras el programa no esté corriendo.
- Existen dos casos: objetos “streamed” (corrientes de bytes a ser enviados a otra máquina) y “persistentes” (objetos alojados en el disco).

El Lenguaje de programación Java

Tipos Primitivos

Constituyen un caso especial:

- No se crea la variable usando new por eficiencia (new aloja en el heap)
- Se crea una variable automática que no es una referencia y asigna el valor en la pila, lo cual es más eficiente.
- Java determina el tamaño de cada tipo primitivo, es invariante de una máquina a otra.

Tipo Primitivo	Tamaño	Mínimo	Máximo	Tipo Wrapper
boolean	-	-	-	Boolean
char	16-bit	Unicode 0	Unicode 216-1	Character
byte	8-bit	-128	+127	Byte
short	16-bit	-215	+215-1	Short
int	32-bit	-231	+231-1	Integer
long	64-bit	-263	+263-1	Long
float	32-bit	IEEE754	IEEE754	Float
double	64-bit	IEEE754	IEEE754	Double
void	-	-	-	Void

El Lenguaje de programación Java

- El tamaño de boolean no está definido; debe ser capaz de tomar los valores literales true o false.
- Los tipos “Wrapper” tienen clases para ellos; esto permite crear un objeto no primitivo en el heap.

Números de alta precisión

Para ejecutar aritmética de alta precisión, Java provee dos clases:

BigInteger y BigDecimal: no tienen tipos primitivos análogos.

Expresiones de Java

- Una sentencia de Java puede ser un comentario, un bloque, una declaración, una expresión o una sentencia de control.
- La mayoría termina con “;” (excepto bloques y comentarios)

Comentarios

Están definidos tres tipos de comentarios:

1. Estilo C++: dos barras “//” en una línea
2. Estilo C: usa /* para indicar principio y */ para final del comentario
3. Definido por Java sólo para documentación: /** */

El Lenguaje de programación Java

javadoc genera una página HTML para cada archivo JAVA, y una más para el paquete como un todo. Para cada archivo .java, javadoc registra el nombre de la clase y la signatura completa de cada método.
Para hacer visible un comentario en javadoc, se usa /**....*/

```
/**
* Sirve para hacer una prueba inicial del ambiente.
*
* @author Martin
* @version 1.0
*/
public class HolaMundo
{
    public static void main(String[] args)
    {
        System.out.println("Hola, mundo!\n");
    }
}
```

La salida será:
Hola, mundo!

El Lenguaje de programación Java

Si se genera documentación, se obtiene una página HTML como la siguiente:

Class HolaMundo

java.lang.Object
    HolaMundo

```
public class HolaMundo
extends Object
```

Sirve para hacer una prueba inicial del ambiente.

Version:

1.0

Author:

Martin

Sigue.....

El Lenguaje de programación Java

Declaración de variables

Todas las variables locales deben ser declaradas antes de ser usadas. La declaración es de la forma:

```
int nInt;       // se declara una variable
int nInt = 10; // se declara una variable y se la inicializa
```

Ámbito

- Este concepto determina la visibilidad y ciclo de vida de los nombres definidos.
- En C, C++ y Java, el ámbito está determinado por la presencia de llaves {}.

Ej.:

```
{
 int x = 12;//sólo se dispone de x
  {
    int q = 34; //se dispone de ambos: x y q
  }
// sólo se dispone de x, q fuera del ámbito
}
```

Una variable definida dentro de un ámbito está disponible sólo hasta el final del bloque dentro del cual se la define.

No se admite en Java definir dos variables con el mismo nombre en bloques de código “anidados”.

El Lenguaje de programación Java

Ámbito de los objetos

Los objetos en Java no tienen el mismo ciclo de vida que las primitivas. Cuando se crea un objeto en Java usando new, el mismo sigue existiendo aún después del final del ámbito donde fue creado. Ej:

```
{
  String s = new String("materia");
} // fin del ámbito
```

- La referencia s finaliza su ciclo de vida al final del bloque.
- El objeto String que s referenciaba sigue ocupando memoria.
- Java tiene un “garbagge collector” que busca todos los objetos que no vuelven a referenciarse y libera la memoria ocupada por ellos para que pueda ser utilizada nuevamente.

El Lenguaje de programación Java

Operadores

Para enteros y Punto flotante

De asignación: =, +=, -=, *=, /=
Comparación: ==, !=
Unario de signo: -
Aritméticos: +, -, *, /
Incremento y Decremento: ++, --
Bitwise: >>, <<, ~, &, |, ^

Booleans

De asignación: =
Comparación: ==, !=
Lógico NOT: ! (convierte true a false y viceversa)
AND: & o &&
OR: | o ||
XOR: ^
Ternario If-Then_Else: (?:)

El Lenguaje de programación Java

Las Clases en Java

- Los usuarios pueden definir sus propios tipos para los objetos que use el programa, utilizando la palabra clave *class*
- Esta es la única forma que aporta Java para definir nuevos tipos, ya que no posee *typedef, struct o enum* como el lenguaje C.

La definición de un tipo de usuario tiene la forma:

```
class MiClase
{
 // componentes de la clase
}
```

Esta definición introduce un nuevo tipo.

El Lenguaje de programación Java

Los objetos de ese tipo se crean utilizando la palabra clave *new*.

```
MiClase obj = new MiClase();
```

Todo lo que se hace en Java es "Definir clases, crear objetos de ese tipo y enviarles mensajes".

El Lenguaje de programación Java

- Una clase tiene dos tipos de integrantes: datos miembro, que llamaremos campos y funciones miembro (término de C++) que en Java llamaremos métodos.

- Los campos se utilizan para describir las propiedades de los datos de la clase; se utilizan las reglas de las variables locales.

Un campo puede ser:
- Un objeto de cualquier tipo;
- Un tipo primitivo (se puede inicializar en el punto de definición en la clase)

Ej. de una clase CuentaBancaria:

```
class CuentaBancaria
{
 double saldo_actual;  //saldo de la cuenta
 int ncta;  //número de la cuenta
}
```

El Lenguaje de programación Java

Esta clase no hace nada pero se puede crear un objeto con ella.

```
CuentaBancaria cuenta = new CuentaBancaria();
```

Se pueden asignar valores a los datos miembro. Para referirse a un dato miembro se utiliza el operador “.”

referenciaObjeto.member

Para asignar un valor, se escribe:

cuenta.ncta = 1025;

Cuando un campo es un tipo primitivo, Java garantiza valores por default para ese tipo si no se inicializa.
Los valores default son: false para el tipo boolean, null para char, 0 para byte, short, int, 0L para long, 0.0f para float y 0.0d para double.

El Lenguaje de programación Java

Métodos (Funciones miembro de clase)

En Java los métodos determinan los mensajes que un objeto puede recibir. Las partes fundamentales de un método son: nombre, argumento, tipo de retorno y cuerpo

```
tipoRetorno nombreMetodo (/*lista de argumentos*/)
{
 /* cuerpo del método */
}
```

El Lenguaje de programación Java

Ej.

```
class CuentaBancaria {
 // propiedades de los campos
 double saldoActual;  //saldo de la cuenta
 int nCta;  //número de la cuenta

 // métodos
 void deposito(double cant) // realización de un depósito
 {
    if (cant >=0.0)
    {
     saldoActual += cant;
    }
 }

 void retiro(double cant)
 {
    if(cant >= 0.0 && cant < saldoActual)
    {
     saldoActual -= cant;
    }
 }
}
```

El Lenguaje de programación Java

Los métodos:

- Pueden ser creados como parte de una clase.
- Pueden ser invocados sólo para un objeto y ese objeto debe ser capaz de ejecutar el método de la llamada.
- Se utiliza la notación:
  nombreObjeto.nombreMetodo(arg1,..., argn).
- El acto de llamada de un método se conoce como "enviar un mensaje a un objeto".
- Los argumentos de la llamada pueden ser cualquier tipo de objetos, de modo que en la lista de argumentos debe especificarse correctamente el tipo del objeto que se pasa al método y su nombre.

El Lenguaje de programación Java

En Java, los métodos se deben definir dentro de una clase. No es admisible separar la definición de la declaración.

- Al invocar el método, se pasa el control a la llave "{".
- Se sale del método por la llave de cierre o por la sentencia *return*.
- Si se devuelve algo diferente de void, el valor se devuelve con *return* seguido de una expresión.

Ej.:

```
boolean flag( ) {return true;}
void noDevuelve( ) { return; }
void noDevuelve2 ( ) { }
int devInt( ) { return 4;}
```

El Lenguaje de programación Java

Diferencia entre clase y objeto

- Una clase describe un tipo de cosa.
- Ej. Un televisor es un aparato que se puede representar por la *clase* TV.
- Un televisor en particular, por ej. el mío es un *objeto* de la clase TV.

```
TV mitv = new TV();
```

El objeto mitv es la referencia a una instancia de la clase TV.
Una clase puede tener cualquier número de objetos.

El Lenguaje de programación Java

Usando el operador "." se accede a las variables de instancia y a los métodos de un objeto.

```
class TV
{
    int nCanal; // número del canal actual

    int setCanal(int nuevoCanal)  // cambia el canal
    {
        if (nuevoCanal >1 && nuevoCanal < 60)
        {
            nCanal = nuevoCanal;
        }
        return nCanal;
    }
}
```

El Lenguaje de programación Java

- Los programas comunes portables de Java se conocen como "*aplicaciones*", los que corrían con ayuda de otro programa (un Web browser) se conocían como "*applets*".

Applets

- Los applets surgieron por la necesidad de los diseñadores Web que requerían páginas "dinámicas" capaces de interactuar con el usuario. Esta capacidad no era provista por los lenguajes de la época.
- Para que un applet se ejecutara en un navegador era necesario que el navegador implementara la tecnología NPAPI, que fue retirada de los navegadores principales a mediados del año 2015.

El Lenguaje de programación Java

Construcción de aplicaciones

1. Ingresar el código en un archivo de texto
2. Compilar las clases con la opción Compile (javac)
3. Ejecutar la aplicación (java)

Analizando el código del ejemplo anterior:

```
public class HolaMundo
{
   public static void main(String[] args)
   {
      System.out.println("Hola, mundo!");
   }
}
```

El Lenguaje de programación Java

Observamos:

1. La clase es declarada como pública

- Hay distintos niveles de acceso: public, private, protected.
- Cualquiera puede acceder a clases y miembros de clases que son públicos.
- Cuando se escribe una aplicación: se necesita declarar una clase principal de la aplicación como pública.
- El entorno de Java ejecuta el método main() de esa clase.

2. La clase contiene un método de nombre main()

- El método main() es de clase, no de instancia.
- Los métodos de clase se declaran como static.

3. El método main() se declara como public static y retorna void

- En Java se define una clase como el punto de comienzo de la aplicación.
- El usuario llama el método main() de esa clase y el trabajo comienza desde allí.
- En Java cada dato y cada método (incluso main()) pertenece a una clase.

El Lenguaje de programación Java

Paso de argumentos

Cuando se define main(), se escribe:

...main(String[ ] args)

- Java incorpora cada argumento de la línea de comandos en un String y pasa el arreglo de Strings a main().
- Se puede luego utilizar args[0], args[1], etc.
- Se pude conocer la cantidad de argumentos utilizando args.length()
- Existe una librería de clases (package) que se incluye automáticamente en forma implícita a cada archivo java: *java.lang*
- El paquete java.lang posee un objeto System. Uno de sus campos es un objeto out de tipo static PrintStream.
- El objeto out encapsula el concepto de "standard output"; uno de sus métodos es println()

El Lenguaje de programación Java

Las Aplicaciones java pueden tener interfaz gráfica, el JFC (java Fundation Classes) provee dos paquetes awt y swing para el desarrollo de aplicaciones gráficas.
Ejemplo:

```
import java.awt.*;
import javax.swing.*;
import java.awt.event.*;

public class MiFrame extends JFrame
{//se extiende la clase JFrame del paquete javax.swing.JFrame
  //defino referencias a los componentes gráficos del Frame
  private JTextField jtfTexto;
  private JButton boton;


  public MiFrame()//constructor
  {
     super("Frame ejemplo");
     jtfTexto=new JTextField(10); //crea un campo de texto de longitud 10
     boton=new JButton("OK");//crea un boton con etiqueta "OK"
     Container cp=getContentPane(); //obtiene una referencia al contenedor de JFrame
     cp.setLayout(new FlowLayout());//determina como se distribuyen los componentes
     cp.add(jtfTexto);//agrega el componente al contenedor
     cp.add(boton);//agrega el componente al contenedor
     boton.addActionListener(new AL()); //para "escuchar" los eventos sobre el boton

  }
```

El Lenguaje de programación Java

```
  private class AL implements ActionListener
 {//actionListener que maneja los eventos sobre el boton
    public void actionPerformed(ActionEvent evt)
    {
            jtfTexto.setText("Ahora si");
    }
  }

  public static void main(String []args)//programa principal
  {
    MiFrame mf=new MiFrame(); //crea objeto de tipo MiFrame
    mf.setSize(200,300);//se establece el tamaño del frame
    mf.setVisible(true);//muestra el frame
  }
}
```

JFrame es un contenedor de alto nivel (una ventana) que contiene el resto de los componentes gráficos, una aplicación java gráfica, debe definir al menos un contenedor de alto nivel.